

Datenbanken und Web-Techniken

Databases and Web Techniques



Abteilung Datenmanagement
Prof. Dr. Michael Martin
Florian Hahn

Organisation - Contacts



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ



Prof. Dr. Michael Martin

Professor

Telefon: +49 371 531-35312

Fax: +49 371 531-835312

E-Mail: [michael.martin@...](mailto:michael.martin@tu-chemnitz.de)

Adresse: Straße der Nationen 62, 09111 Chemnitz

Raum: A12.336.2 (alt: 1/336B)

Florian Hahn

Wissenschaftlicher Mitarbeiter, Doktorand

Telefon: +49 371 531-35007

Fax: +49 371 531-835007

E-Mail: [florian.hahn@...](mailto:florian.hahn@tu-chemnitz.de)

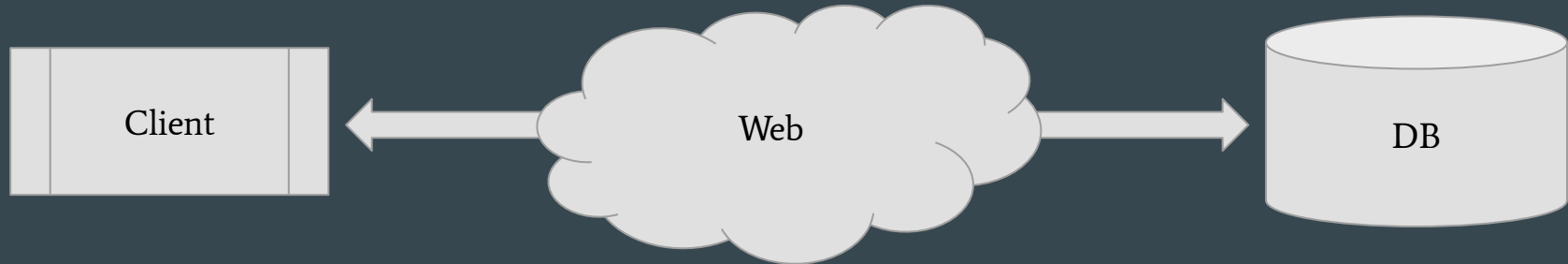
Adresse: Straße der Nationen 62, 09111 Chemnitz

Raum: A12.336.3 (alt: 1/336C)



- Please register in the OPAL Group, linked via DMS website
 - <https://www.tu-chemnitz.de/informatik/dm/>
 - <https://bildungsportal.sachsen.de/opal/auth/RepositoryEntry/297435137>
 - organising resources:
 - slides
 - communication
 - submissions
 - scheduling of presentation slots

- Did you ever tried to access a database from the web?



- (theoretical) background of particular technologies to access databases (over web)
- introduction to web development
- selecting available technologies according to their purpose
- architecture concepts and principles
 - high level
 - details are given in exercises
 - self studies
- Prerequisites
 - basic knowledge about databases (i.e. SQL) and network technologies (OSI)

- Setup and Deployment of SQL- and NOSQL - databases (MySQL, MongoDB)
- Communication between client, server and DB
 - REST
 - APIs
 - GraphQL
- Frontend and Backend development (React.js, Node.js, Express.js)
 - coding examples and tasks
- All slides will be available in OPAL afterwards

- Lecture (5 units + task presentation)
 - Lecture is given in German and afterwards in english
 - Slides will be in English and German
 - Thursdays from 09th of April to 21th of May at 3:30 pm and afterwards 05:15 pm
- Exercises (3x7 units)
 - are given in English and German
 - German Exercise Monday 09:15 am
 - English Exercise Monday 11:30 am and Wednesday 11:30 am
- Project work
 - in English and German
 - see next slide
- During project work
 - Lecture slot (15:30 - 17:00) will be an Q&A session until the end of the project (mixed languages)

- practical project task
 - submission of a term paper
 - presentation
- one task for all students
 - individual assignment
 - no groups are allowed
 - The scope depends on the respective study regulations
- Schedule
 - 21th of May (lecture slot) : presentation of the project task
 - 25th of June (midnight): deadline of project and term paper submission
 - 26th of June(morning 09:00): registration for oral defense slots
 - 6th of July to 17th of July project presentations

Attention: Don't forget to register the exam in Central Examination Office / SB Service and OPAL

Projects from the past (DBW 2024)



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

- Public administrations need examples of how published government data can be used
- Examples DBW 2024:
 - Integration of datasets into a web application currently under development for searching for location-based facilities:
 - Schulen (Grundschule, Oberschule, Förderschule, Gymnasium, Berufsbildende Schule, ...)
 - <https://portal-chemnitz.opendata.arcgis.com/datasets/chemnitz::schulen/about>
 - Kindertageseinrichtungen
 - <https://portal-chemnitz.opendata.arcgis.com/datasets/chemnitz::kindertageseinrichtungen/about>
 - Schulsozialarbeit
 - <https://portal-chemnitz.opendata.arcgis.com/datasets/chemnitz::schulsozialarbeit/about>
 - Jugendberufshilfe
 - <https://portal-chemnitz.opendata.arcgis.com/datasets/chemnitz::jugendberufshilfen/about>
- Examples DBW 2025:
 - Integration of datasets from open source due to important public places for the “Kulturhauptstadt”

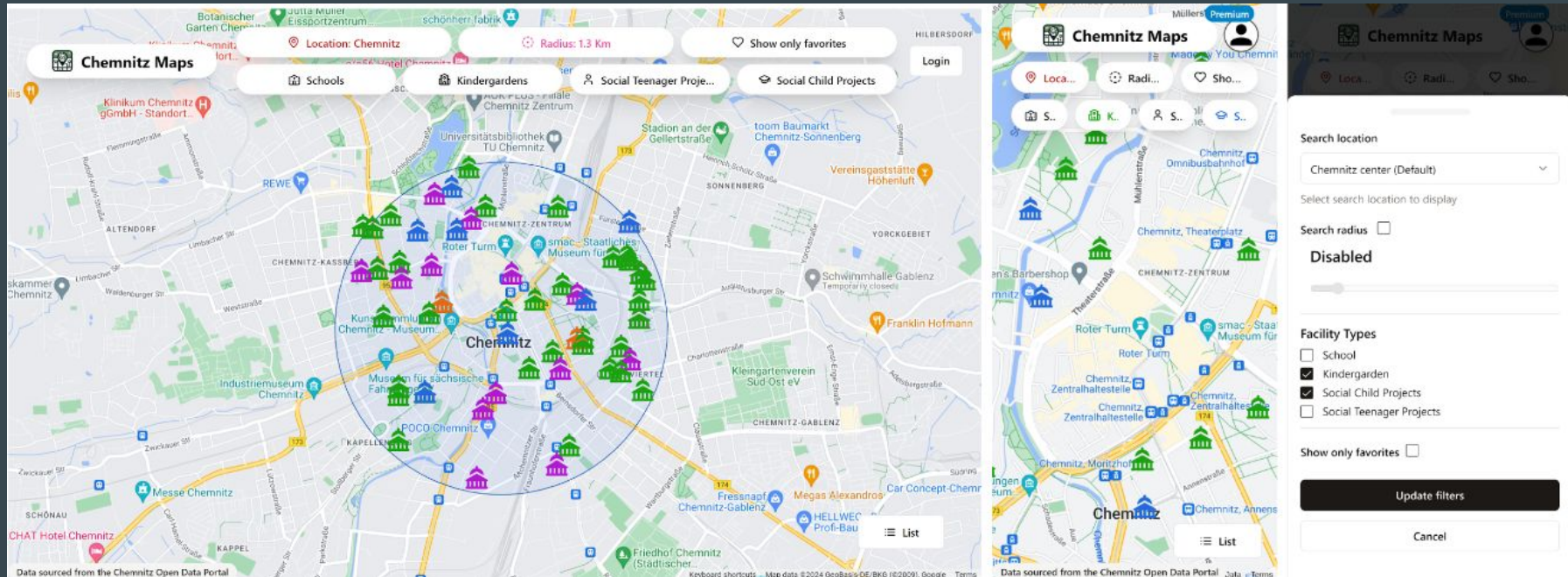
Beispiele aus studentischen Projekten im Kurs DBW 2024



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

- Ca. 70 Projekte in Einzel- und Gruppenarbeiten geschaffen und vorgestellt

Screenshots entnommen aus Hausarbeiten der Studierenden



2015

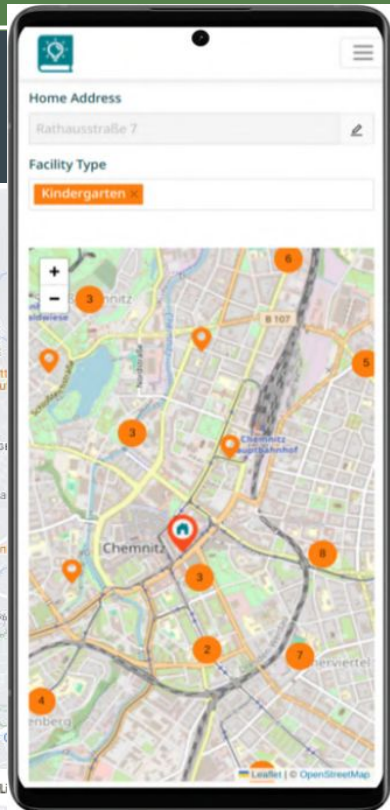

MapMyPlace

 Home
 Map

Filter Panel
Reset Filters

☒ Social Child Projects
☐ Social Teenager Projects
☐ kinderGardens
☐ Schools





Data sourced from the Chemnitz Open Data Portal

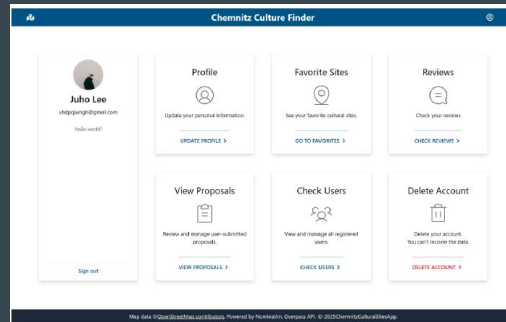
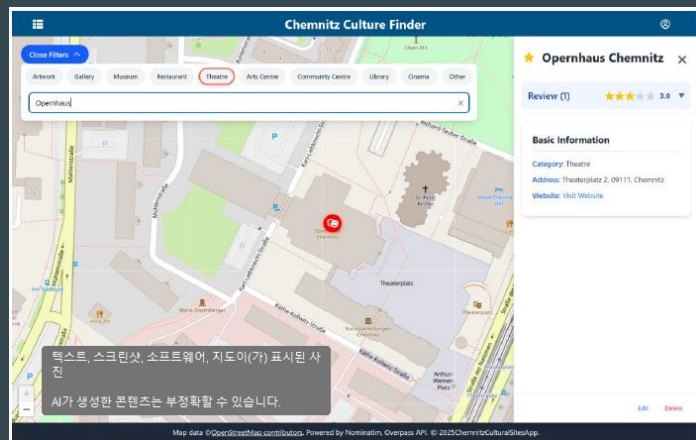
Data sourced from the Chemnitz Open Data Portal [Data](#) [Terms](#)

Beispiele aus studentischen Projekten im Kurs DBW 2025



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

Screenshots entnommen aus Hausarbeiten der Studierenden



- Ian Sommerville
- Software Engineering (Pearson) - 10th. Edition
2018
- <https://katalog.bibliothek.tu-chemnitz.de/Record/0-1641948019>



- Chris Northwood: The Full Stack Developer
<https://doi.org/10.1007/978-1-4842-4152-3>
- Thomas Valentine: Database-Driven Web Development
<https://doi.org/10.1007/978-1-4842-5970-2>
- Frank Zammetti: Modern Full-Stack Development
<https://doi.org/10.1007/978-1-4842-8811-5>

Q&A about organizational things



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

Q&A

Datenbanken und Web-Techniken

Databases and Web Techniques

Lecture 1 - System/Software Architectures



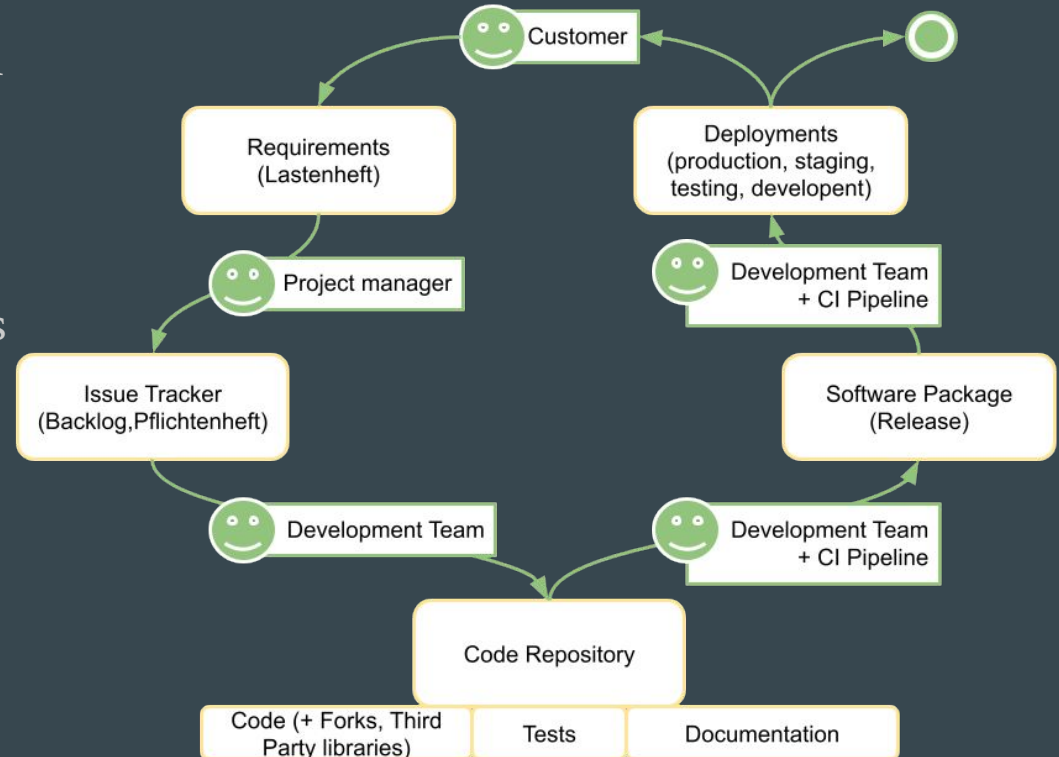
Abteilung Datenmanagement
Prof. Dr. Michael Martin
Florian Hahn

- How would you design an (web-)application that facilitates a database. i.e.:
 - a shop system (mobile / web),
 - an interactive information system (wiki-like) or
 - a web frontend for a set of decentralized information spaces (ticket system for heterogeneous transport providers)
- First, we need to think about a suitable architecture of the target application.
- ... and sure, how to organize the team ...

System / Software Architectures

Classification

- part of design process in each system / software development
- in agile development processes such as SCRUM it's an ongoing process after an initial design decision



- **Design process for identifying** the **sub-systems** making up a system and the framework for sub-system control and communication is architectural design.
 - The output of this design process is a description of the software architecture.
- **Architectural design**
 - An early stage of the system design process.
 - as well in agile process in the beginning but can be developed incrementally
 - iteration and refactoring of architectures are complex and expensive
 - Represents the link between specification and design processes.
 - Often carried out in parallel with some specification activities.
 - involves identifying major system components and their communications

- **Architecture in the small** is concerned with the architecture of individual programs. At this level, we are concerned with the way that an individual program is decomposed into components.
- **Architecture in the large** is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components. These enterprise systems are distributed over different computers, which may be owned and managed by different companies.

- Simple, informal block diagrams showing entities and relationships are the most frequently used method for documenting software architectures.
- But these have been criticised because they lack semantics, do not show the types of relationships between entities nor the visible properties of entities in the architecture.
- Depends on the use of architectural models. The requirements for model semantics depends on how the models are used.
- Box and line diagrams are very abstract - they do not show the nature of component relationships nor the externally visible properties of the sub-systems.
 - However, useful for communication with stakeholders and for project planning.

Advantages of explicit architecture

- **Stakeholder communication**
 - Architecture may be used as a focus of discussion by system stakeholders.
- **System analysis**
 - Means that analysis of whether the system can meet its non-functional requirements is possible.
- **Large-scale reuse**
 - The architecture may be reusable across a range of systems
 - Product-line architectures may be developed.

- As a way of facilitating discussion about the system design
 - A high-level architectural view of a system is useful for communication with system stakeholders and project planning because it is not cluttered with detail. Stakeholders can relate to it and understand an abstract view of the system. They can then discuss the system as a whole without being confused by detail.
- As a way of documenting an architecture that has been designed
 - The aim here is to produce a complete system model that shows the different components in a system, their interfaces and their connections.

- Is there a generic application architecture that can be used?
- How will the system be distributed?
- What architectural styles are appropriate?
- What approach will be used to structure the system?
- How will the system be decomposed into modules?
- What control strategy should be used?
- How will the architectural design be evaluated?
- How should the architecture be documented?



Systems in the same domain often have similar architectures that reflect domain concepts.

- **Performance**
 - Localise critical operations and minimise communications. Use large rather than fine-grain components.
- **Security**
 - Use a layered architecture with critical assets in the inner layers.
- **Safety**
 - Localise safety-critical features in a small number of sub-systems.
- **Availability**
 - Include redundant components and mechanisms for fault tolerance.
- **Maintainability**
 - Use fine-grain, replaceable components.

- A **logical view** , which shows the key abstractions in the system as objects or object classes.
- A **process view** , which shows how, at run-time, the system is composed of interacting processes.
- A **development view** , which shows how the software is decomposed for development.
- A **physical view** , which shows the system hardware and how software components are distributed across the processors in the system.
- Related using use cases or scenarios (+1)

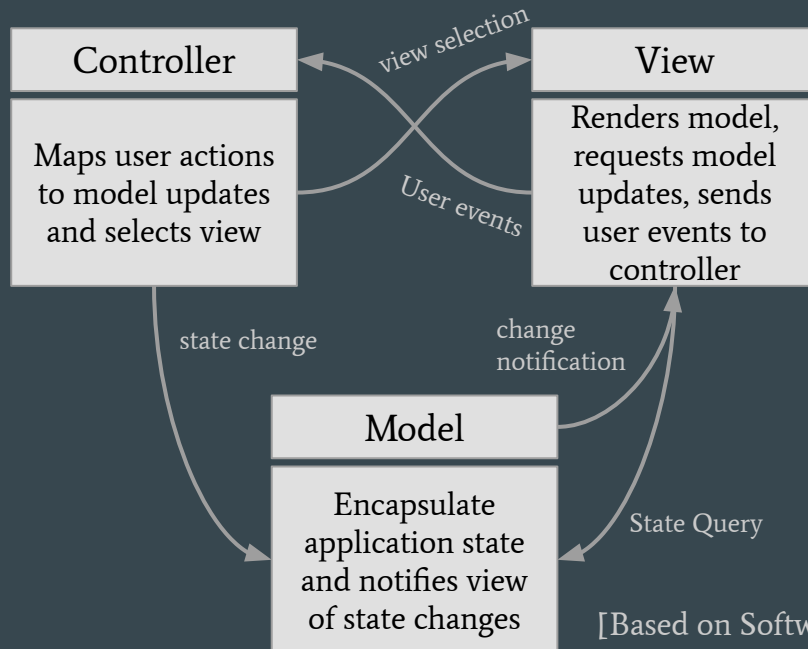
- Pattern are a means of representing, sharing and reusing knowledge.
- An architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.
- Pattern should include information about when they are and when they are not useful.
- Pattern may be represented using tabular and graphical descriptions.

System/Software Architectures

Basic Pattern: MVC



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ



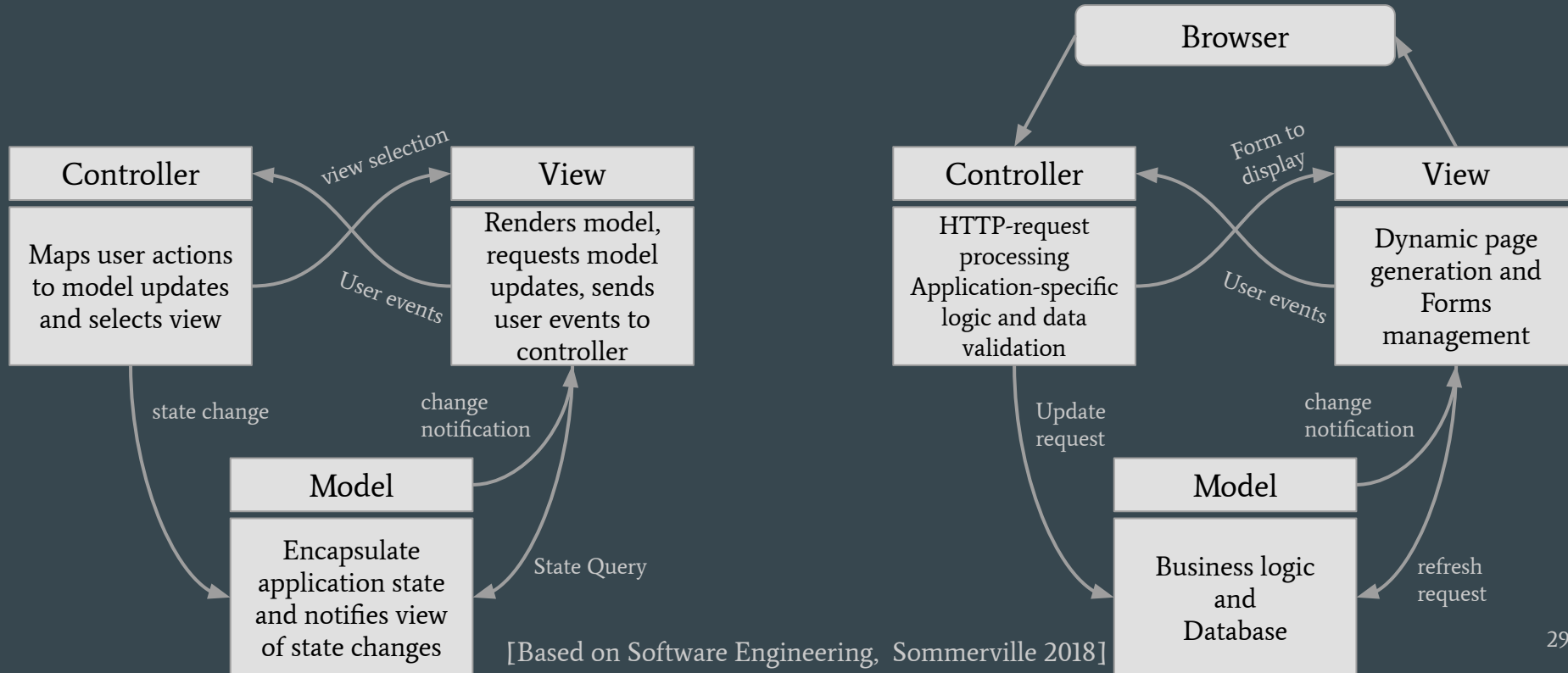
[Based on Software Engineering, Sommerville 2018]

System/Software Architectures

Basic Pattern: MVC adapted to web applications

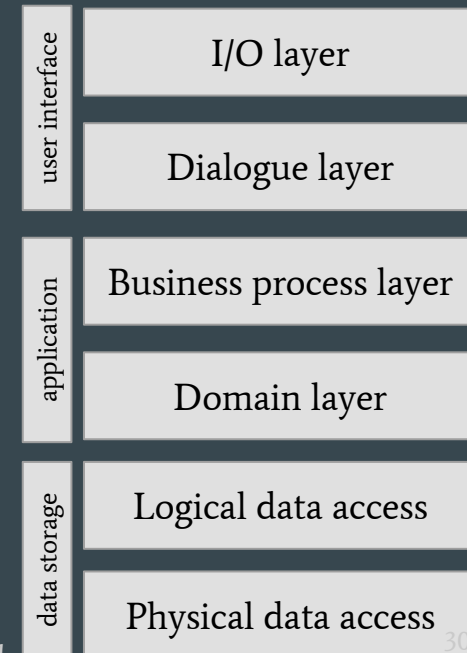


TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ



- logical layer
 - In order to distribute an application, it needs to be divided into layers.
 - The layers of an application are also referred to as "logical layers."
- physical layer
 - Logical layers represent the modularization units of an application.
 - Physical layers represent the units that are distributed across different machines.
- distribution pattern
 - logical layers can be distributed using distribution pattern

logical layer architecture



System/Software Architectures

X-Tier architectures: Monolithic architecture



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

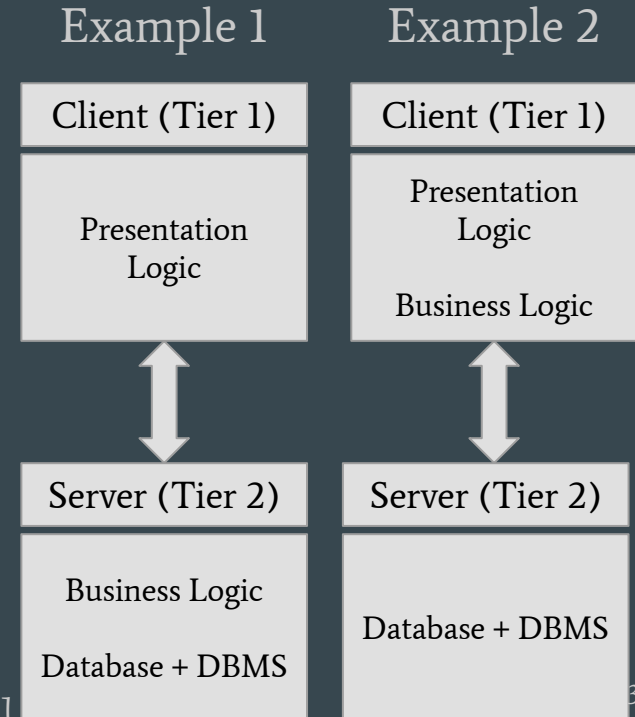
- traditional architecture
 - entire application is developed as a single unit
- three main components:
 - user interface
 - business logic
 - data storage
- components of the application are tightly coupled and interdependent
- simple to develop and deploy (i.e. single executable file)
- difficult to scale and maintain as the application grows



X-Tier architectures: Client-Server (Two-Tier) architecture

- divides an application into two components (tiers), which communicates through a network
- client tier
 - handles user interactions, displays the results and communicates with the server
 - can be standalone app, a web browser or mobile device
- server tier (application tier, backend)
 - processes the requests received from client
 - performs the required operations and returns result to client
 - contains the database
 - can be a physical server or a cluster of servers
 - can be located on-premise (self-hosted) or in the cloud

[Based on Software Engineering, Sommerville 2018]



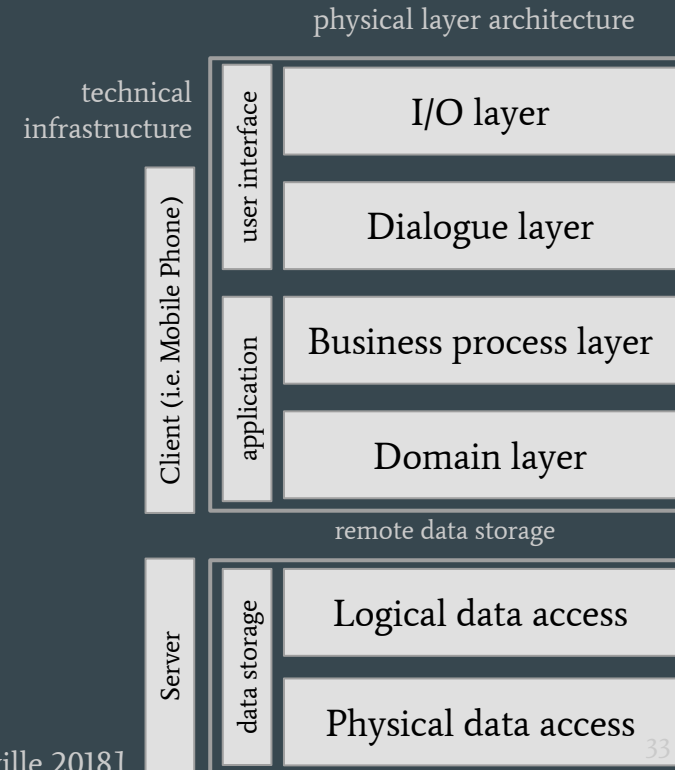
System/Software Architectures

X-Tier architectures: Client-Server (Two-Tier) architecture



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

- Advantages
 - simplicity / ease of development
 - performance
- Disadvantages / Limitations
 - client and server are tightly coupled
 - scaling, maintaining and updating the application can be challenging
- Summary
 - two-tier architectures are a simple way to develop client-server applications
 - they may not be suitable for larger and more complex applications that require high scalability and flexibility



System/Software Architectures

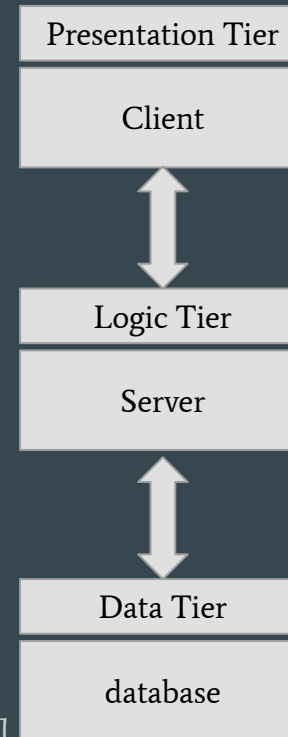
X-Tier architectures: Three-Tier architecture



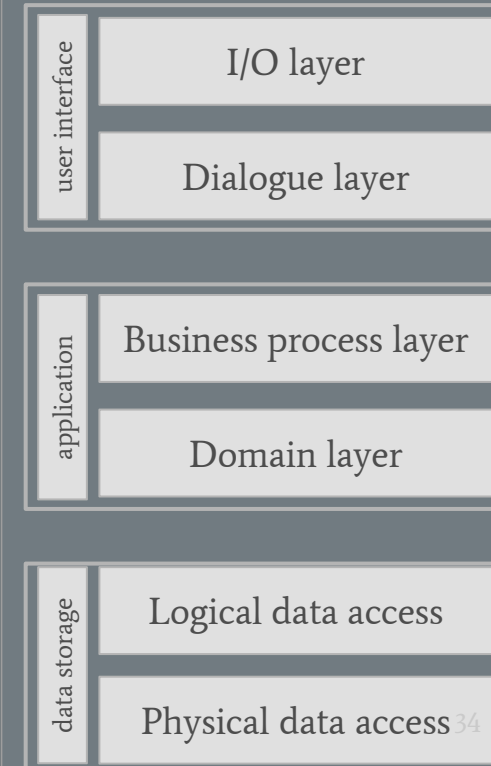
TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

- Presentation Tier (client)
 - interaction with user (I/O)
 - includes UI components
 - communicates with the application tier
- Application Tier (middle tier, logic tier, server)
 - contains the business logic and processes user requests
 - interacts with all tiers
- Data Tier (back-end tier, database)
 - stores and retrieves data
 - includes the database management systems, file servers or other data storage systems
 - communicates with application tier

Three-Tier architecture



logical layer architecture

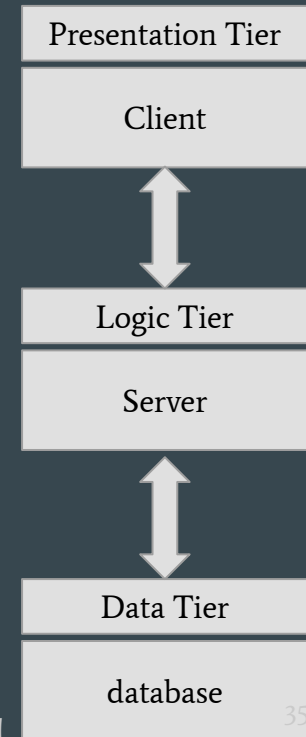


[Based on Software Engineering, Sommerville 2018]

- Advantages

- Scalability
 - each tier can be scaled independently, improved changeability
- Maintainability
 - testing, debugging of tiers separately
- Security
 - use of different security metrics for each tier
- Flexibility
 - changes can be made to one tier without affecting the others
 - improved agility in development process
- Reusability
 - each tier can be reused in different applications

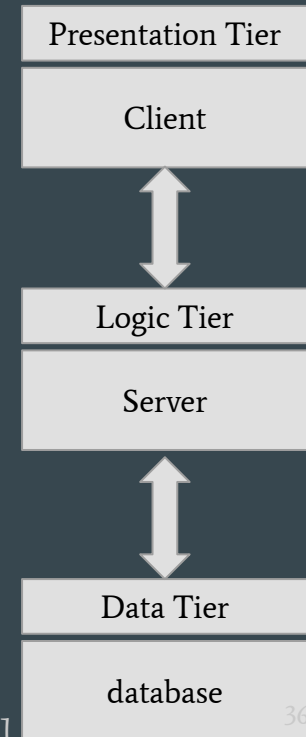
Three-Tier architecture



- Disadvantages / Drawbacks

- Complexity
 - increasing complexity
 - requires additional infrastructure and resources to maintain layers
 - development, deployment and maintenance can be more difficult and time-consuming
- Latency
 - communication between layers can introduce latency or delay, especially if layers are at different physical infrastructures
- Cost
 - can be more expensive than other architectures because of additional hardware, software

Three-Tier architecture



X-Tier architectures: Multi-Tier architectures

- divides an application into multiple logical layers
- each layer has a specific function and level of abstraction
- the Three-Tier architecture is the most common form of multi-tier architectures
- Additional tiers:
 - caching tier
 - can store frequently accessed data to reduce cost expensive data access
 - integration tier
 - integrates different applications or systems
 - web service tier
 - provides access to the application through web services

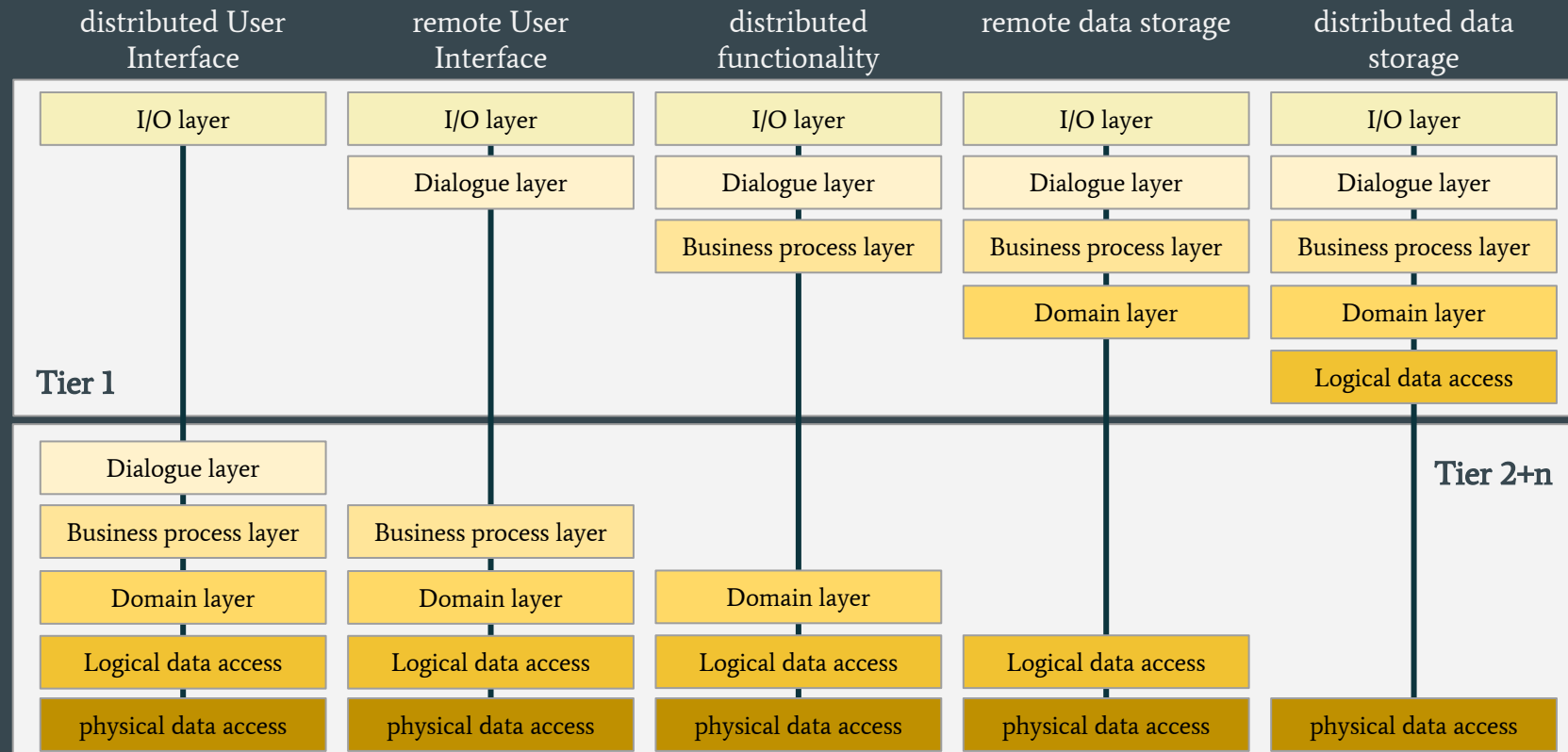
System/Software Architectures



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

X-Tier architectures: distributed architectures

[Based on Software Engineering, Sommerville 2018]



- Systems / Applications are drilled down into multiple small and independent services
- each service run in its own process and communicates with other services using APIs
- Key principles
 - Single responsibility principle
 - Each microservices is responsible for a single business capability or function
 - Decentralized data management
 - Each microservice manages its own data and has its own data store
 - reduced dependencies between services

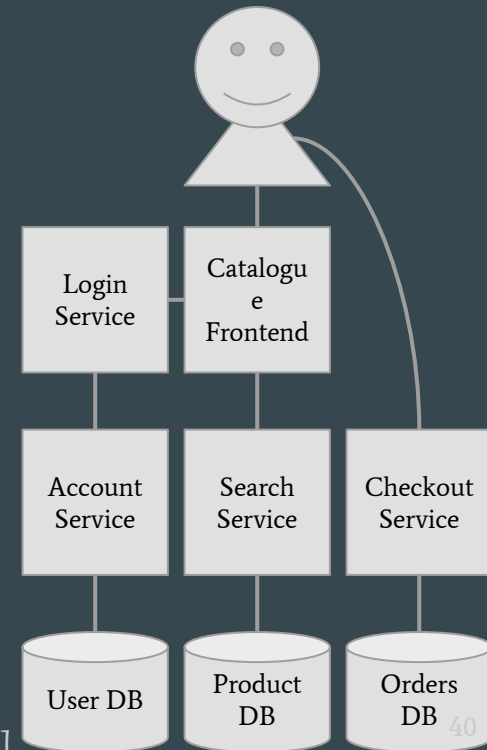
System/Software Architectures

Microservice Architecture



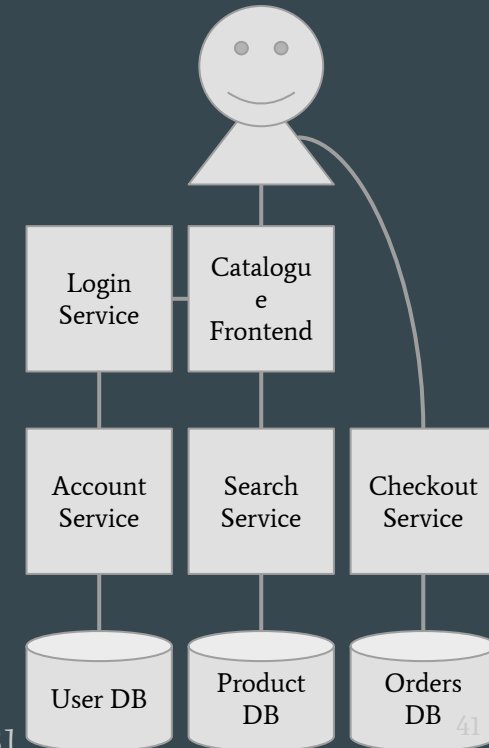
TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

- Autonomous teams
 - each microservice is developed, tested and deployed independently by a dedicated team
 - improved agility
 - faster time-to-market
- API-based communication
 - Microservices communicate with each other through well defined APIs
 - enable loose coupling and improves flexibility
- Containerization
 - Microservices are typically deployed in containers
 - enables efficient resource utilization and makes it easy to manage and deploy services



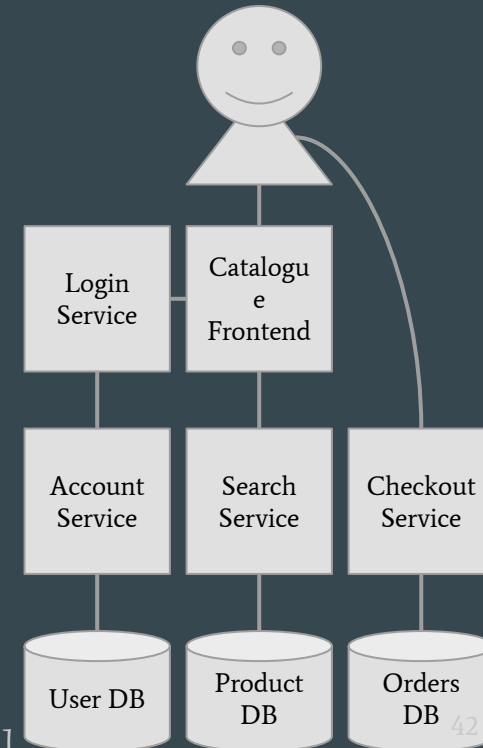
[Based on Software Engineering, Sommerville 2018]

- Advantages
 - Scalability
 - independent scaling
 - changes can be handled on demand more efficiently
 - Flexibility
 - independent development and deployment
 - faster response by changing business requirements
 - Resilience
 - fault-tolerant design
 - a failure of one service does not affect the entire application
 - Technology heterogeneity
 - can be developed using heterogeneous technologies, programming languages, technical environments
 - Reusability
 - services can be reused in different applications



- Disadvantages / Drawbacks

- Increased complexity / team discipline
 - Microservice architecture adds complexity to the application
 - principles have to be taken into account by everyone (i.e. documentation, carry-its-own-data-principle)
- Distribution
 - additional challenges around communication, monitoring and management
- Overhead
 - Microservices requires additional infrastructure and resources
- Testing
 - Testing can be more fine-grained (advantage), but there are more components that have to be monitored by system architects and product owners



- Monoliths and Microservices
 - Two extremes of architecture pattern
 - The truth / reality is somewhere in between
- Multi-tier / Three-tier architectures
 - flexible and scalable architecture that can handle large amounts of data and user requests
 - is widely used in web applications

System/Software Architectures

The End



TECHNISCHE UNIVERSITÄT
IN DER KULTURHAUPTSTADT EUROPAS
CHEMNITZ

